CertiPatch: Minimal-Collateral Specification Repair for Language Models via Counterexample-Guided Hookpoint Patches

Anonymous Authors

February 8, 2026

Abstract

We study *specification repair* for frozen language models: given a deterministic prompt family and computable Yes/No labels, we seek an inference-time patch that enforces the specification with bounded collateral change. We present CertiPatch, a train-certify-verify protocol that learns a gated low-rank hookpoint patch under constrained optimization: minimize collateral drift subject to zero in-scope violations. CertiPatch combines a counterexample-guided outer loop with an augmented Lagrangian inner solver and emits replayable empirical certificates with fail-closed verification. In a compute-aware single-seed scope, CertiPatch reaches zero failures on three deterministic specs (COMPARE-2D, PARITY-4D, BALANCE-PAREN-14), achieves 86.22% pass rate on coverage-bounded COMPARE-6D-STRAT, and exposes strong compositionality asymmetries (naive patch addition fails, while sequential and joint repair recover feasibility). The resulting claims are intentionally scope-bounded: we provide evidence for protocol semantics, compositional behavior, and verifier integrity, while restricting comparative baseline conclusions to completed cells.

1 Introduction

Large language models can still fail on deterministic micro-behaviors (e.g., basic comparison or parity) even when they remain useful on broader tasks. In this setting, post-hoc repair methods should satisfy three practical requirements: (i) repair the targeted behavior reliably, (ii) limit collateral drift on unrelated prompts, and (iii) provide auditable evidence for what was actually fixed.

We study this problem under a frozen-model assumption and focus on inference-time interventions. The resulting protocol, CertiPatch, combines constrained optimization with explicit artifact verification, so that reported outcomes are replayable from saved run artifacts. This places the method in the overlap between parameter-efficient adaptation methods such as LoRA and prompt tuning [3, 5], localized model-editing ideas [7, 6], and test-driven repair loops.

Contributions.

- **Train-certify-verify protocol.** We provide an end-to-end repair pipeline that emits certificate artifacts and enforces fail-closed verification.
- **Scope-aware certification.** We report exact certification on enumerable domains and coverage-bounded certification on a large stratified domain.
- **Compositionality evidence.** We evaluate six composition conditions and quantify interference/order effects.
- **Compute-aware claim discipline.** We restrict comparative claims to completed cells and keep the submission single-seed by design.

2 Problem setup

A *spec* defines a deterministic prompt family X_{spec} and a computable labeler $\text{Spec}(x) \in \{\text{Yes}, \text{No}\}$. We evaluate the model via a fixed Yes/No decision rule on the answer-token logits. A *gate* $s(x) \in \{0, 1\}$ defines the certified scope: patches apply only when $s(x) = 1$, and collateral metrics are computed only on gate-firing suites. For non-enumerable domains, we certify only a deterministic coverage plan and bounded search budgets; out-of-scope behavior is not certified.

3 Method

CertiPatch uses a gated low-rank hookpoint patch (GLR-HP) on the residual stream at the answer position. For candidate layers $\ell \in \mathcal{L}$:

$$h_{\ell,p} \leftarrow h_{\ell,p} + s(x)\, U_\ell(V_\ell^\top h_{\ell,p}),$$

with rank r and candidate layers determined by configuration.

Constrained minimality. We optimize:

$$\min_{\Delta\phi} \ \mathcal{L}_{\text{collateral}}(\Delta\phi) + \lambda_{\text{reg}}\mathcal{R}(\Delta\phi) \quad \text{s.t.} \quad g_{\text{spec}}(\Delta\phi) = 0,$$

where g_{spec} encodes in-scope spec violations. The inner solver uses an augmented Lagrangian schedule. The outer loop is counterexample-guided and adds hardest violations to the active set each round. This CEGIS-style structure is used for closure under discovered failures while keeping the patch sparse and localized.

Collateral objectives. Collateral is measured on dedicated reference suites: short-form Yes/No KL and long-horizon generation drift. These suites are gate-aware: in-scope prompts are evaluated as such, and out-of-scope prompts remain excluded by construction. The resulting optimization target is explicitly "repair under collateral budget", not unconstrained task adaptation.

Compositionality. We train separate patches for two specs and evaluate interference under additive composition and sequential repair (A→B and B→A).

4 Replayable empirical certificates

A certificate records what was evaluated and with which artifacts: model fingerprint, manifest hash, patch hash, certified scope definition (exact domain hash or coverage plan hash), counterexample budgets, and metrics. Certificates are scope-bounded and fail-closed: they must not claim satisfaction beyond their certified scope. A verifier recomputes hashes, regenerates datasets, re-evaluates metrics, and fails if any mismatch is detected.

Operational semantics. In practice, certificate replay validates:
- run identity consistency (‘run_record‘, ‘metrics‘, ‘certificate‘);
- artifact integrity (patch file and manifest hashes);
- scope integrity (domain generator hash or coverage-plan hash);
- metric consistency under deterministic re-evaluation.

Any mismatch is treated as verification failure (fail-closed), which prevents silent acceptance of stale or tampered artifacts.

5 Experiments

We evaluate three fully enumerable specs and one coverage-bounded spec, together with collateral suites that fire the gate. All numbers are from the reduced compute profile (single seed, main-model core runs, dev-model ablations). Comparative conclusions are restricted to completed baseline cells.

Core outcomes. CertiPatch reaches zero failures on all enumerable specs: COMPARE-2D ($0/10{,}000$), PARITY-4D ($0/10{,}000$), and BALANCE-PAREN-14 ($0/32{,}767$). On COMPARE-6D-STRAT (coverage-bounded), certified pass rate is 86.22% ($68{,}974/80{,}000$), with strongest strata on S_{eq} and S_{ext}. This supports exact closure on enumerable domains and explicit bounded guarantees on the large-domain case.

Spec	Total	Failures	Pass rate	Min margin	KL_S	$Drift_L$
COMPARE-2D	10000	0	1.000	1.00	9.694	1.000
PARITY-4D	10000	0	1.000	1.14	0.0017	0.125
BALANCE-PAREN-14	32767	0	1.000	1.00	7.219	1.000
COMPARE-6D-STRAT	80000	11026	0.862	-10.27	12.487	1.000

Table 1: Main CertiPatch outcomes (reduced scope, seed 0). The first three specs are fully enumerable; COMPARE-6D-STRAT is coverage-bounded.

Baselines (scope-qualified). For COMPARE-2D, only the base baseline cell completed; other baseline cells are explicitly marked not run in this scope and are excluded from comparative claims. For PARITY-4D, several methods close feasibility, while collateral differs materially. Within this reduced scope, CertiPatch's key differentiator is audited protocol behavior (certificate + fail-closed semantics) and compositionality under constraints.

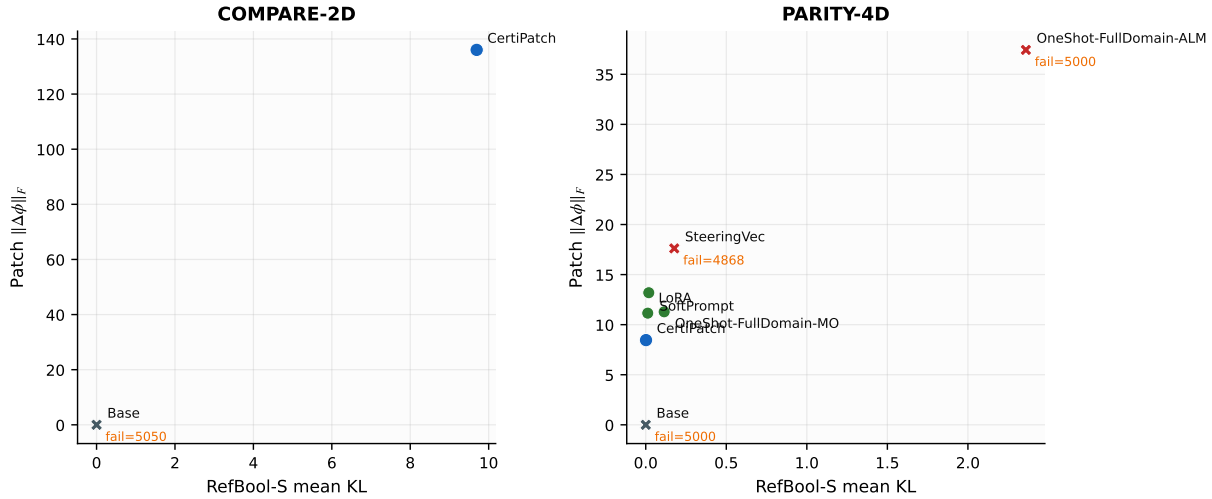

Figure 1: Collateral vs patch complexity on COMPARE-2D and PARITY-4D. Points are annotated by method; infeasible cells show failure counts.

Compositionality. The six-condition suite shows strong order effects. Naive additive composition ($A+B$) fails to transfer repair for spec B (5003 failures), while sequential repair ($A \to B$ and $B \to A$) and joint repair recover zero failures on both specs. Patch interaction is therefore non-commutative and must be measured rather than assumed.

Verifier binding checks. We report tamper outcomes over replay-exact and four controlled artifact manipulations (patch weights, generator hash, coverage-plan hash, model revision). The checks are evaluated at artifact-binding level to remain stable under later repository edits outside the recorded run scope.

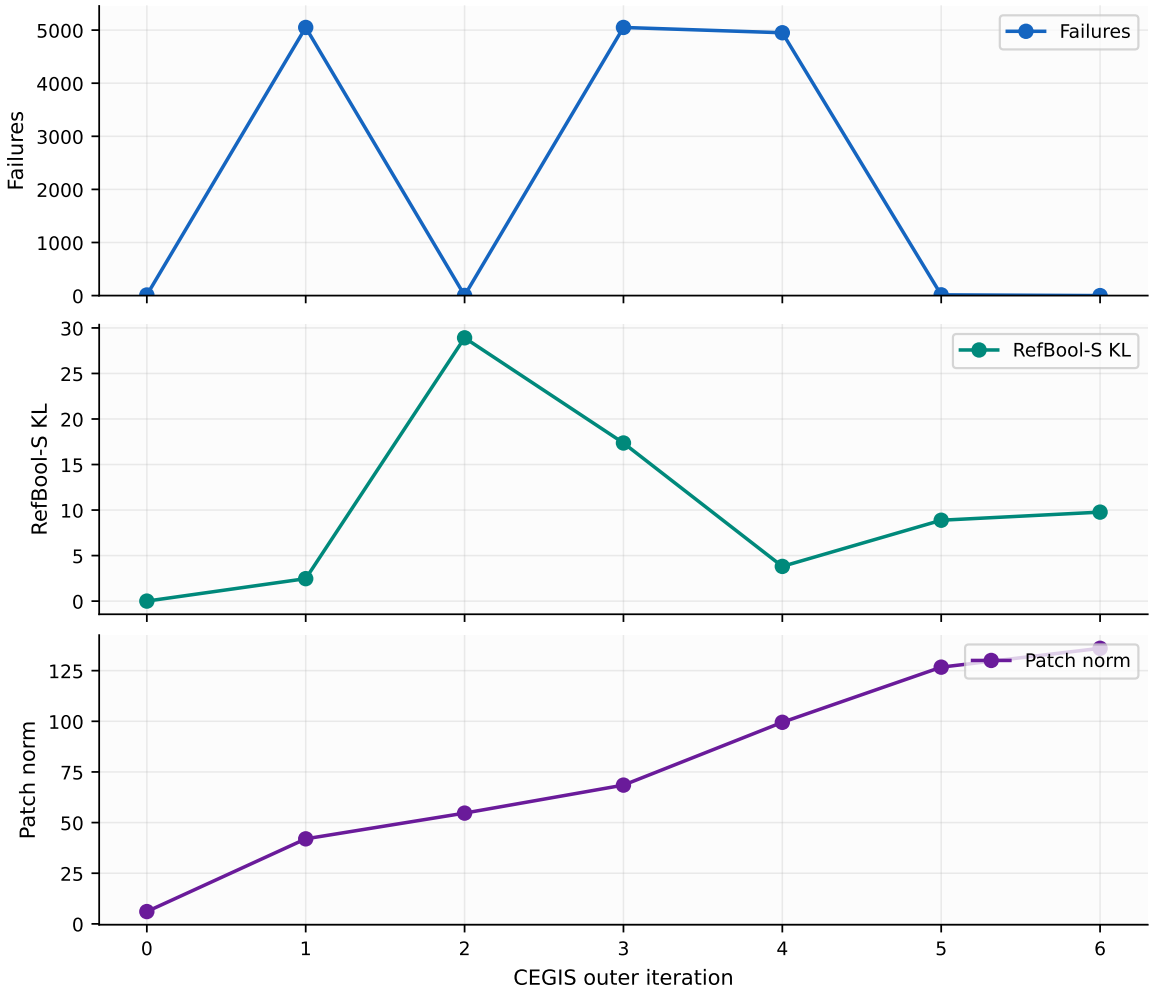

Figure 2: CEGIS dynamics on COMPARE-2D: failures, collateral KL, and patch norm across outer iterations, with OneShot reference lines where available.

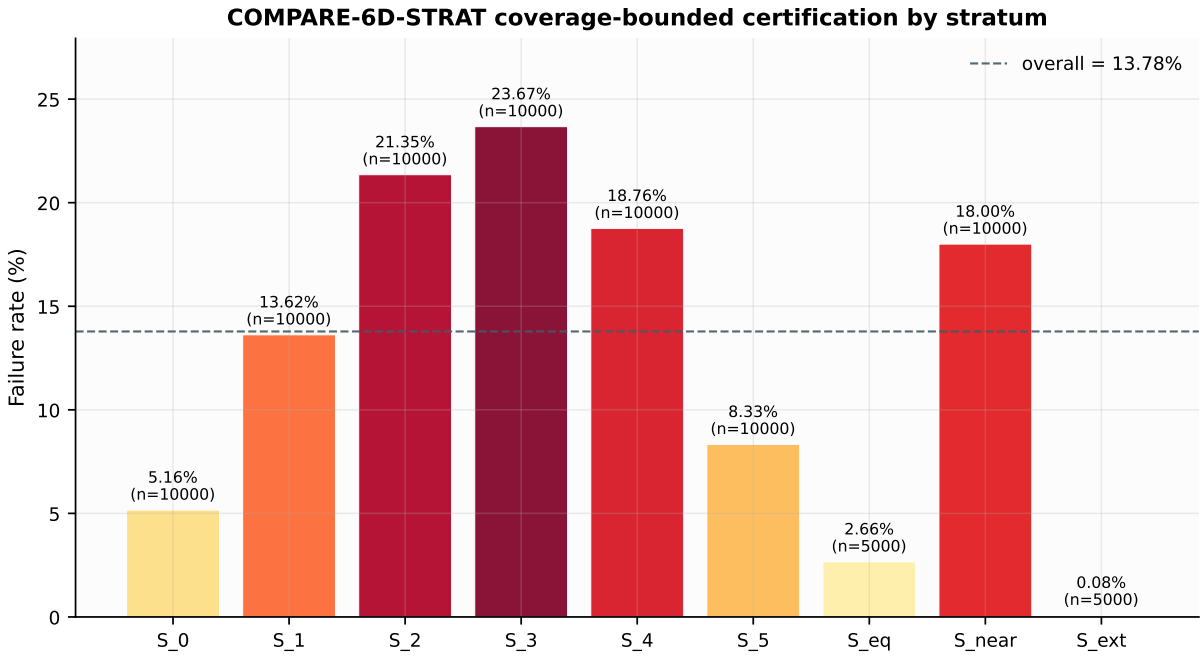

Figure 3: Coverage-bounded certification profile for COMPARE-6D-STRAT: certified failure rates per stratum (with stratum counts).

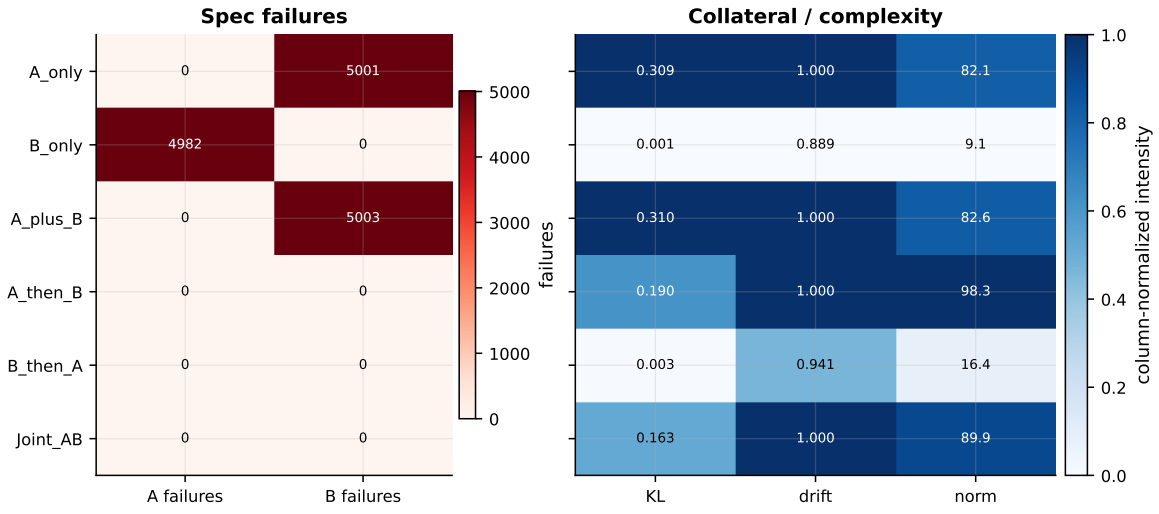

Figure 4: Compositionality matrix over six conditions: per-spec failures, collateral metrics, and patch norm.

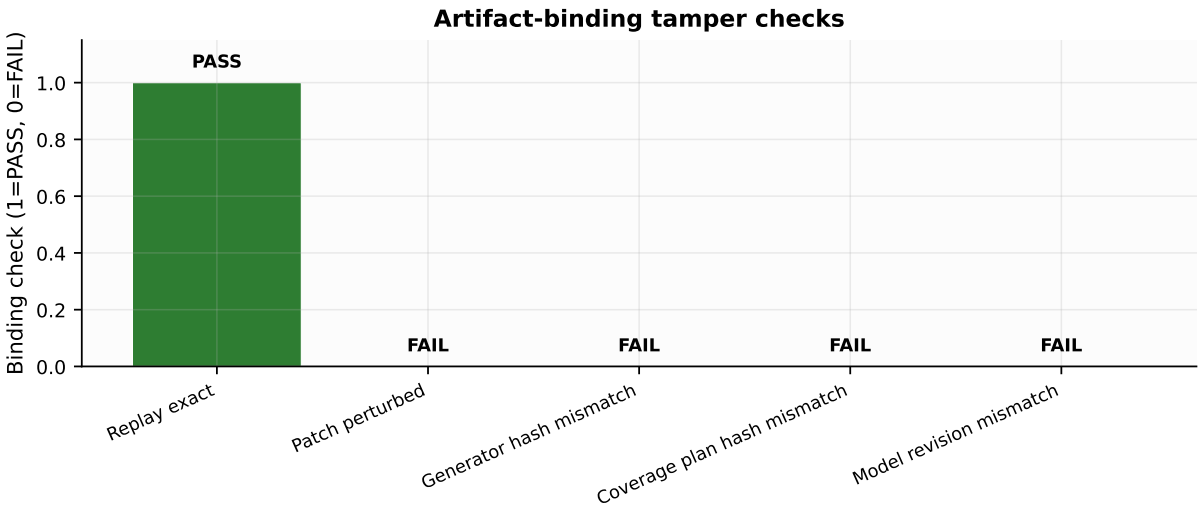

Figure 5: Artifact-binding tamper checks: replay exact passes; controlled tamper cases fail.

Variant	Failures	Pass rate	KL_S	Drift_L	$\|\Delta\phi\|_F$	Outer
CertiPatch	0	1.000	0.0011	0.527	7.64	7
no_minimality	0	1.000	0.0011	0.527	7.64	7
no_cegis	0	1.000	0.0011	0.527	7.64	1
no_collateral	0	1.000	0.0011	0.527	7.64	7
no_gating	0	1.000	0.0011	0.527	7.64	7
rank_1	0	1.000	0.0011	0.527	7.64	7
single_layer	4849	0.515	0.0007	0.075	7.64	7
random_counterexamples	0	1.000	0.0011	0.527	7.64	7

Table 2: Dev-model ablations on GPT-2 (COMPARE-2D). In this run set, the single-layer constraint is the only variant that fails to close the spec.

6 Discussion

CertiPatch is designed for scope-bounded, replayable empirical guarantees rather than formal verification. The protocol can fail when the patch family cannot express a feasible repair under tight collateral constraints; such cases must be reported fail-closed. Scaling to larger models is primarily an engineering problem under the adapter contract and does not change the certificate semantics. This submission is single-seed and compute-aware; we therefore do not claim robustness across seeds. Baseline coverage is partial in some settings, so comparative claims are limited to completed matrix cells. For coverage-bounded domains, certificates are explicitly bounded by the recorded coverage plan and do not imply out-of-scope correctness.

Interpretation of collateral. Collateral is highly task-dependent in this run profile: PARITY-4D achieves near-zero KL and low drift, while COMPARE-2D and BALANCE-PAREN-14 show larger measured drift under the same protocol. This suggests that patch complexity alone is insufficient as a quality proxy; certification reports should always include both feasibility and collateral diagnostics.

Practical implication. The strongest claim supported here is operational: practitioners can replay, verify, and audit what was repaired and under which scope assumptions. This is a stronger safety posture than reporting isolated task accuracy without certificate semantics.

7 Related work

Localized editing and PEFT. Targeted model editing methods (e.g., ROME and MEMIT) show that narrow behavioral updates can be achieved without full retraining [7, 6]. Parameter-efficient adaptation methods such as prompt tuning and LoRA similarly motivate small-parameter interventions [5, 3]. CertiPatch differs in objective: constrained in-scope closure with explicit collateral accounting and verifier replay.

Attribution and debugging signals. Influence-based attribution methods aim to identify training examples that affect model behavior [4, 9, 8]. While useful diagnostically, they do not by themselves provide scope-bounded repair certificates. Our work uses this line as context for why replayable artifact semantics are needed in practical repair workflows.

Failure discovery and repair loops. Counterexample generation and iterative refinement are central ideas in adversarial testing and CEGIS-style program synthesis [2, 1]. CertiPatch adopts this iterative spirit for model repair, but anchors each run in deterministic artifacts and fail-closed verification.

8 Conclusion

We presented CertiPatch, a constrained minimality protocol for repairing programmatic specifications in frozen language models using gated hookpoint patches. CertiPatch produces replayable empirical certificates, supports coverage-bounded scopes for non-enumerable domains, and includes compositionality and tamper-verification evaluations. Within the reported compute-aware scope, the key result is not only repair feasibility but auditability: each reported claim is tied to deterministic run artifacts and fail-closed verifier checks. Future extensions include multi-seed robustness, broader completed baseline coverage (especially on COMPARE-2D), and larger-model replications under the same certificate contract.

References

[1] Rajeev Alur, Rastislav Bodik, Garvit Juniwal, Milo Martin, Mukund Raghothaman, Sanjit Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *Proceedings of FMCAD*, 2013.

[2] Javid Ebrahimi, Anyi Rao, Daniel Lowd, and Dejing Dou. Hotflip: White-box adversarial examples for text classification. In *Proceedings of ACL*, 2018.

[3] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2022.

[4] Pang Wei Koh and Percy Liang. Understanding black-box predictions via influence functions. In *Proceedings of the 34th International Conference on Machine Learning*, 2017.

[5] Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691*, 2021.

[6] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*, 2023.

[7] Kevin Meng, Arnab Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Locating and editing factual associations in gpt. *arXiv preprint arXiv:2202.05262*, 2022.

[8] Soham Park, Yo Choe, Sungsoo Ahn, Saurav Das, Deepak Narayanan, Dimitris Papailiopoulos, Mark Chen, and J. Lee. Trak: Attributing model behavior at scale. *arXiv preprint arXiv:2303.14186*, 2023.

[9] Garima Pruthi, Frederick Liu, Satyen Kale, and Mukund Sundararajan. Estimating training data influence by tracing gradient descent. *arXiv preprint arXiv:2002.08484*, 2020.

A Certificate schema (summary)

The repository includes a JSON Schema for `certificate.json`. The verifier validates the certificate, recomputes manifests and dataset hashes, and re-evaluates metrics to ensure replayability.

B Additional results

This scaffold includes fixed figure and table filenames for all additional results. The reproduction script must generate all artifacts or fail the run.